In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
dataset=pd.get_dummies(dataset,drop_first=True)
print(dataset)
x=dataset[['age','bmi','children','sex_male','smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset[['charges']].ravel()
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [ ]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [24]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
param_grid={
    "criterion":['squared_error','absolute_error','poisson'],
    'max_features':['sqrt','log2'],
    'n_estimators':[10,100]
}
grid=GridSearchCV(RandomForestRegressor(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\base.py:1403: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().
  return fit_method(estimator, *args, **kwargs)
```

Out[24]:

▸ GridSearchCV
▸ best_estimator_: RandomForestRegressor
▸ RandomForestRegressor

In [ ]:
```
grid_pred=grid.predict(x_test)
print("prediction data:",grid_pred)
print("actual data:",y_test)
```

In [26]:
```
from sklearn.metrics import r2_score
result=r2_score(grid_pred,y_test)
result
```

Out[26]:

```
0.10014519146478629
```

In [27]:
```
re=grid.cv_results_
print("r_score of best parameter{}:".format(grid.best_params_),result)
```

```
r_score of best parameter{'criterion': 'poisson', 'max_features': 'log2', 'n_estimators': 100}: 0.10014519146478629
```

In [28]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[28]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_n_estimators	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.049928	0.012107	0.008556	0.003646	squared_error	sqrt	10	{'criterion': 'squared_error', 'max_features':...	0.796618	0.836715	0.770167	0.747612	0.798151	0.789853	0.029941	
1	0.520552	0.102802	0.026899	0.009053	squared_error	sqrt	100	{'criterion': 'squared_error', 'max_features':...	0.826991	0.891856	0.801031	0.770358	0.826508	0.823349	0.040056	
2	0.046281	0.019029	0.009313	0.005671	squared_error	log2	10	{'criterion': 'squared_error', 'max_features':...	0.819946	0.877237	0.770064	0.748872	0.815264	0.806276	0.044519	
3	0.331510	0.005878	0.024573	0.002425	squared_error	log2	100	{'criterion': 'squared_error', 'max_features':...	0.827008	0.899964	0.810339	0.781926	0.841312	0.832110	0.039261	
4	0.069250	0.009209	0.005595	0.002600	absolute_error	sqrt	10	{'criterion': 'absolute_error', 'max_features'...	0.811293	0.868661	0.776291	0.740022	0.773544	0.793962	0.043632	
5	0.616470	0.012877	0.021819	0.002103	absolute_error	sqrt	100	{'criterion': 'absolute_error', 'max_features'...	0.826681	0.891308	0.796395	0.776353	0.824147	0.822977	0.038896	
6	0.071761	0.002187	0.004397	0.000551	absolute_error	log2	10	{'criterion': 'absolute_error', 'max_features'...	0.810515	0.881049	0.762347	0.777911	0.789132	0.804191	0.041510	
7	0.650082	0.059575	0.021372	0.002372	absolute_error	log2	100	{'criterion': 'absolute_error', 'max_features'...	0.828252	0.901697	0.809840	0.782973	0.842594	0.833071	0.039681	
8	0.029434	0.000804	0.003910	0.000253	poisson	sqrt	10	{'criterion': 'poisson', 'max_features': 'sqrt...	0.786889	0.868971	0.769616	0.752938	0.784938	0.792670	0.040063	
9	0.251572	0.001734	0.019498	0.000487	poisson	sqrt	100	{'criterion': 'poisson', 'max_features': 'sqrt...	0.825863	0.886140	0.794821	0.768311	0.821871	0.819401	0.039284	
10	0.029455	0.000814	0.003959	0.000294	poisson	log2	10	{'criterion': 'poisson', 'max_features': 'log2...	0.829576	0.869373	0.781763	0.766995	0.824411	0.814424	0.036510	
11	0.250871	0.022722	0.016671	0.003547	poisson	log2	100	{'criterion': 'poisson', 'max_features': 'log2...	0.828228	0.903952	0.813634	0.779748	0.845052	0.834123	0.041004	

In [ ]: